

Package ‘rfsnEfficiency’

July 1, 2026

Title Risk-Reduction Efficacy and Efficiency of Risk-Management Configurations

Version 0.0.0.9000

Description A general, on-demand toolkit for evaluating risk-management or safety-net configurations. From user-supplied end-of-season outcome draws (ideally one element per agent) and program parameters (cost, premium, subsidy, and an indemnification rule that may be a function), it builds a configurable outcome (revenue, profit, margin, or a custom / cost-side measure) relative to a no-program baseline and computes efficacy scores (mean lift, variability reduction) and a risk-reduction efficiency ratio, with comparisons across regimes or scenarios. Implements the risk-reduction-efficiency metric of Tsiboe et al. (2025).

License GPL (>= 3)

Encoding UTF-8

LazyData true

Imports data.table,
stats,
utils

Suggests testthat (>= 3.0.0),
piggyback,
usethis,
knitr,
rmarkdown

VignetteBuilder knitr

Config/testthat/edition 3

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 3.5)

Contents

build_agent_outcomes	2
build_outcome	3
compute_efficiency_scores	4
efficiency_score_crosswalk	5
income_transfer_score	5
regime_deltas	6

risk_reduction_efficiency	6
run_efficiency_analysis	7
simulate_example_outcomes	8
summarise_scores	9
toy_outcomes	9
trim_mean	10
variability_reduction_score	11

Index	12
--------------	-----------

build_agent_outcomes	<i>Build per-agent baseline/treated outcomes from end-of-season draws + parameters</i>
----------------------	--

Description

The flexible entry point: hand in each agent's underlying end-of-season outcomes and the program parameters, and get back stacked baseline/treated outcome draws ready for `compute_efficiency_scores()`.

Usage

```
build_agent_outcomes(
  agents,
  indemnity,
  outcome = "outcome",
  cost = 0,
  premium = 0,
  subsidy = 0,
  agent_id_col = "agent_id",
  extra_cols = NULL,
  floor_at = 0
)
```

Arguments

agents	a list with one element per agent. Each element is either a numeric vector of the base end-of-season outcome (draws), or a data.frame/data.table of draws (rows = crop years / Monte Carlo draws) containing at least the outcome column. List names, if present, become agent ids.
indemnity	indemnification rule: a function(agent_df) returning a per-draw indemnity vector, OR a column name in each agent's data, OR a numeric scalar/vector. Indemnity is ADDED to the treated outcome.
outcome	name of the base end-of-season outcome column (default "outcome"); ignored when an agent element is a plain numeric vector.
cost, premium, subsidy	program parameters, each a function(agent_df), a column name, or a numeric scalar/vector. cost is subtracted from the base outcome (enters both baseline and treated). premium is the producer-paid premium, subtracted from the treated outcome. subsidy is carried through for program-context summaries (not added to the outcome).
agent_id_col	name of the agent id column in the output (default "agent_id").

extra_cols	optional character vector of additional per-draw columns to carry through from each agent's data (e.g., "commodity_code", "combination", "regime", "commodity_year").
floor_at	numeric floor applied to both outcome series (default 0; NA = none).

Details

$$outcome_baseline = outcome - cost$$

$$outcome_treated = outcome_baseline + indemnity - producer_premium$$

Value

a data.table stacked over agents with agent_id, draw, outcome_baseline, outcome_treated, and the resolved cost, producer_premium, subsidy, indemnity columns (plus extra_cols).

Examples

```
# Two agents, each with 100 end-of-season revenue draws; indemnity is a
# function paying 70% of the shortfall below a 600 guarantee.
set.seed(1)
agents <- list(
  cornA = data.frame(outcome = pmax(0, rnorm(100, 600, 150))),
  cornB = data.frame(outcome = pmax(0, rnorm(100, 550, 200)))
)
indem <- function(df) pmax(0, 600 - df$outcome) * 0.70
d <- build_agent_outcomes(agents, indemnity = indem, premium = 25, subsidy = 0.62)
compute_efficiency_scores(d, by = "agent_id")[, .(agent_id, mean_index, risk_index, efficiency)]
```

build_outcome

Construct the analysis outcome and its no-safety-net baseline

Description

Construct the analysis outcome and its no-safety-net baseline

Usage

```
build_outcome(
  data,
  base_value = "revenue",
  cost = NULL,
  transfers = "indemnity",
  cost_protection = NULL,
  premium = "producer_premium",
  floor_at = 0
)
```

Arguments

data	data.frame/data.table with the component columns.
base_value	column name of the gross pre-safety-net outcome (e.g., "revenue"; for profit pass revenue plus a cost).
cost	optional column subtracted from base_value. May be a stochastic per-observation series so cost risk enters the outcome (enabling cost-side products). NULL = revenue; a production-cost column = profit.
transfers	character vector of revenue/yield-side payout columns ADDED to the treated outcome (e.g., c("indemnity", "title1_payment")). NULL for products that provide no revenue-side protection.
cost_protection	character vector of cost-side payout columns (a product that pays when costs rise) ADDED to the treated outcome. NULL if none.
premium	column SUBTRACTED from the treated outcome (producer-paid premium for whichever product(s) are active); NULL to ignore.
floor_at	numeric floor applied to both series (default 0; NA = none).

Value

data with new columns outcome_baseline and outcome_treated.

Examples

```
## Not run:
build_outcome(d, base_value = "revenue", cost = NULL)           # revenue
build_outcome(d, base_value = "revenue", cost = "input_cost")   # profit
build_outcome(d, "revenue", cost = "input_cost", transfers = NULL, # cost-only
              cost_protection = "margin_indemnity", premium = "margin_premium")

## End(Not run)
```

compute_efficiency_scores

Compute efficacy and efficiency scores from a generic outcome

Description

Compute efficacy and efficiency scores from a generic outcome

Usage

```
compute_efficiency_scores(
  data,
  by,
  outcome_baseline = "outcome_baseline",
  outcome_treated = "outcome_treated"
)
```

Arguments

data data.frame/data.table with grouping columns and the two outcome series (outcome_baseline, outcome_treated), one row per draw/year within each group.

by character vector of grouping columns (the unit the scores describe, e.g., agent identifiers x combination).

outcome_baseline, outcome_treated column names of the no-safety-net and with-safety-net outcome series.

Value

data.table, one row per by group, with moments (*_base/*_sn), relative indices (*_index), headline scores (mean_index, risk_index, efficiency), flags, percent transforms, and the downside/shortfall flavors.

efficiency_score_crosswalk	<i>Crosswalk: intuitive names <-> legacy / source-pipeline column names</i>
----------------------------	---

Description

Maps the package’s intuitive score names to the terse column names used by earlier implementations of the same metrics (e.g., its, Relcv, sner1).

Usage

efficiency_score_crosswalk()

Value

data.frame(score, legacy, meaning).

income_transfer_score	<i>Income Transfer Score (ITS)</i>
-----------------------	------------------------------------

Description

Ratio of the expected outcome with the safety net to the expected outcome without it. ITS > 1 indicates the configuration raises the mean outcome. (Eq. 2 in Tsiboe et al. 2025.)

Usage

income_transfer_score(treated_mean, baseline_mean)

Arguments

treated_mean, baseline_mean numeric vectors of mean outcomes with and without the safety net.

Value

numeric vector (non-finite ratios returned as NA).

regime_deltas	<i>Regime deltas (alt - ref) for every numeric score</i>
---------------	--

Description

Regime deltas (alt - ref) for every numeric score

Usage

```
regime_deltas(
  scores,
  by,
  metrics = NULL,
  regime_col = "regime",
  ref = NULL,
  alt = NULL
)
```

Arguments

scores	data.table from compute_efficiency_scores() stacked over regimes (must include regime_col).
by	grouping columns identifying a comparable unit across regimes.
metrics	numeric score columns to difference (default: all numeric).
regime_col	regime column name (default "regime").
ref, alt	reference and alternative regime labels. If NULL (default), the first two (sorted) regime values are used.

Value

data.table with by, metric, the two regime columns, and delta.

risk_reduction_efficiency	<i>Risk Reduction Efficiency Ratio (RRER) – Equation (3)</i>
---------------------------	--

Description

$$RRER = \mathbb{I}(VRS < 1) \mathbb{I}(ITS > 1) \frac{1 - VRS}{ITS - 1}$$

Usage

```
risk_reduction_efficiency(its, vrs)
```

Arguments

its	Income Transfer Score (from income_transfer_score()).
vrs	Variability Reduction Score (from variability_reduction_score()).

Details

The proportional variability reduction deflated by the proportional income (outcome) gain required to achieve it. It is 0 unless the configuration both lifts the mean ($ITS > 1$) and cuts variability ($VRS < 1$); otherwise it ranges over $[0, \infty)$, with larger values = more efficient risk mitigation. Exactly Equation (3) of Tsiboe et al. (2025).

Value

numeric vector in $[0, \infty)$.

Examples

```
risk_reduction_efficiency(its = 1.05, vrs = 0.80) # both conditions met
risk_reduction_efficiency(its = 0.98, vrs = 0.80) # mean not lifted -> 0
```

```
run_efficiency_analysis
```

Run an on-demand efficiency analysis

Description

High-level driver: given simulated outcomes (one row per draw/year x group, optionally tagged by regime/scenario), build the outcome if needed, compute the efficacy/efficiency scores per regime, and difference regimes.

Usage

```
run_efficiency_analysis(
  data,
  by,
  regime_col = "regime",
  outcome_args = NULL,
  ref = NULL,
  alt = NULL
)
```

Arguments

data	data.frame/data.table of simulated outcomes. Must contain the by columns and either (outcome_baseline, outcome_treated) or the raw components that build_outcome() needs (then pass outcome_args).
by	character vector of grouping columns (e.g., c("group_id", "scenario")).
regime_col	regime column name (default "regime"); created as "all" if absent.
outcome_args	named list of arguments forwarded to build_outcome() when the outcome columns are not already present (e.g., list(base_value = "outcome", transfers = "indemnity", premium = "premium")).
ref, alt	reference and alternative regime labels for the deltas. If NULL (default) and exactly/at least two regimes are present, the first two (sorted) are used.

Value

list with scores (per regime) and deltas (alt - ref).

Examples

```
# Self-contained: synthetic outcomes, no external data required.
d <- simulate_example_outcomes()
res <- run_efficiency_analysis(
  data = d, by = c("group_id", "scenario"),
  outcome_args = list(base_value = "outcome", transfers = "indemnity",
    premium = "premium"))
head(res$scores)
head(res$deltas)
```

```
simulate_example_outcomes
```

Simulate a tiny example outcomes dataset (no external data required)

Description

Generates synthetic per-draw outcomes for two regimes and a few scenarios so the package can be demonstrated, tested, and learned standalone. This is an ILLUSTRATIVE generator, not a calibrated model – the numbers carry no meaning.

Usage

```
simulate_example_outcomes(
  n_groups = 6,
  n_draws = 200,
  regimes = c("baseline", "reform"),
  seed = 1
)
```

Arguments

n_groups	number of groups (e.g., units / commodities / portfolios).
n_draws	draws (years / Monte Carlo) per group per scenario per regime.
regimes	character vector of regime labels.
seed	RNG seed for reproducibility.

Value

a data.table with group_id, scenario, regime, outcome, indemnity, and premium.

Examples

```
d <- simulate_example_outcomes(n_groups = 3, n_draws = 50)
head(d)
```

summarise_scores	<i>Summarise producer-level scores into robust group summaries</i>
------------------	--

Description

Collapses the per-producer score table from `compute_efficiency_scores()` to one row per by group, reporting robust (trimmed-mean) and median summaries of the three headline metrics plus the share of producers that genuinely reduce risk. A few producers with near-zero baseline revenue produce extreme indices, so the means are trimmed (see [trim_mean](#)); the medians are reported alongside as a robustness check. Every summary accepts an optional weight column, so results can be expressed per acre, per dollar of liability, or per simulation weight rather than per producer.

Usage

```
summarise_scores(dt, by, weight = NULL, lo = 0.005, hi = 0.995)
```

Arguments

dt	data.table of per-producer scores (from <code>compute_efficiency_scores()</code>); must contain <code>mean_index</code> , <code>risk_index</code> , <code>efficiency</code> , and <code>reduces_risk</code> .
by	character vector of grouping columns, or NULL for a single overall summary.
weight	optional name of a numeric weight column in dt. NULL (default) weights every producer equally; when supplied, the trimmed means, medians, and risk-reducing share are all weighted by it.
lo, hi	trimming quantile cut points forwarded to trim_mean .

Value

data.table, one row per by group, with `n_producers`, trimmed-mean `mean_index/risk_index/efficiency`, their `med_*` medians, and `pct_reduces_risk`.

See Also

[compute_efficiency_scores](#), [trim_mean](#)

toy_outcomes	<i>Toy outcomes dataset for examples and CI</i>
--------------	---

Description

A small, committed dataset so examples and the test suite run with no network access. Each insurance pool's calibrated yields *across years* serve as its per-agent outcome draws; the indemnity and premium are illustrative (two regimes, baseline and reform) and carry no policy meaning.

Usage

```
toy_outcomes
```

Format

A data.table with one row per pool x regime x year and columns:

- group_id** insurance-pool identifier (the agent).
- regime** "baseline" or "reform".
- year** commodity year (the draw index).
- outcome** calibrated yield (the end-of-season outcome).
- indemnity** illustrative shortfall payment below the guarantee.
- premium** illustrative producer-paid premium.

Source

Built by data-raw/scripts/build_toy_dataset.R from the [USFarmSafetyNetLab calibrated_yield release](#).

trim_mean	<i>Trimmed (optionally weighted) mean</i>
-----------	---

Description

Mean of x after discarding the tails below the lo and above the hi quantiles. This guards headline score summaries against the handful of producers whose near-zero baseline revenue makes their indices explode and dominate a plain mean (the same trimming used in Tsiboe et al. 2025). Non-finite values are dropped before trimming. The tails are cut at the (unweighted) lo/hi sample quantiles of x – the extreme values are extreme regardless of weight – and, when `weights` is supplied, the retained values are then averaged with those weights. Equal weights therefore reproduce the unweighted trimmed mean exactly.

Usage

```
trim_mean(x, lo = 0.005, hi = 0.995, weights = NULL)
```

Arguments

- `x` numeric vector to summarise.
- `lo, hi` lower/upper quantile cut points (defaults 0.005 / 0.995).
- `weights` optional numeric vector of non-negative weights, the same length as `x`. `NULL` (default) gives the unweighted trimmed mean.

Value

length-one numeric; `NA_real_` when no finite values remain.

Examples

```
trim_mean(c(1, 2, 3, 4, 1000))
trim_mean(c(1, 2, 3, 4, 1000), weights = c(1, 1, 1, 1, 5))
```

`variability_reduction_score`*Variability Reduction Score (VRS)*

Description

Ratio of an outcome-variability measure with the safety net to the measure without it. $VRS < 1$ indicates the configuration reduces variability. Works for ANY variability input (CV, downside semivariance, loss-conditional severity, ...), giving the headline and the robustness flavors.

Usage

```
variability_reduction_score(treated_var, baseline_var)
```

Arguments

`treated_var, baseline_var`
numeric vectors of the variability measure with and without the safety net.

Value

numeric vector (non-finite ratios returned as NA).

Index

* datasets

toy_outcomes, [9](#)

build_agent_outcomes, [2](#)

build_outcome, [3](#)

compute_efficiency_scores, [4](#), [9](#)

efficiency_score_crosswalk, [5](#)

income_transfer_score, [5](#)

regime_deltas, [6](#)

risk_reduction_efficiency, [6](#)

run_efficiency_analysis, [7](#)

simulate_example_outcomes, [8](#)

summarise_scores, [9](#)

toy_outcomes, [9](#)

trim_mean, [9](#), [10](#)

variability_reduction_score, [11](#)